

TRABALHO IA 3: IA GENERATIVA

TEMA 2: *OUTPUT* ESTRUTURADO

Brunna Moura Carlos Cruz Rafael Queiroz Victor
Bitencourt

Universidade do Estado da Bahia (UNEB)

4 de Julho de 2026

- O problema consiste na criação de uma aplicação RAG que consiga retornar *output* estruturado, com mecanismos de validação, mostrando que uma solução de IA pode ser utilizada em sistemas maiores, integrada a fluxos existentes.
- Principal mecanismo utilizado é o *output* estruturado, fornecido nativamente por LLMs hoje em dia.
- Como funciona o *output* estruturado nativo de LLMs?

DOCUMENTO FONTE

Coloque uma imagem da capa do documento aqui!

INFORMAÇÕES

- A base documental consiste no relatório da CPMI de 8 de janeiro de 2023, quando houve tentativa de golpe de Estado no Brasil.
- A base documental foi escolhida estrategicamente, tratando de um assunto possivelmente conhecido por modelos de IA, no entanto, esperando-se no presente trabalho que a mesma traga fontes para sustentar seus resultados, fornecendo uma análise crítica do desempenho do modelo de LLM no fornecimento confiável e embasado de informações.

BIBLIOTECAS UTILIZADAS

Coloque uma imagem das bibliotecas utilizadas aqui!

EXPLICAÇÃO SOBRE AS BIBLIOTECAS

- Carregamento de arquivos: `pymupdf` - um dos melhores carregadores de PDF do mercado, se destacando para PDFs com representação XML presente (*tagged PDFs*). Pela integridade do documento fonte, a biblioteca cumpriu bem sua tarefa. Contras: licença estritamente AGPL.
- Segmentação de *chunks*: `langchain-text-splitters`, utilizando o `RecursiveCharacterTextSplitter`. Chunks de tamanho 2000 com *overlap* de 200.

EXPLICAÇÃO SOBRE AS BIBLIOTECAS

- *Embeddings*: modelo local de redes neurais convolucionais `pt_core_news_lg`, fornecido pela biblioteca `spaCy`, pela leveza em processamento local e dispensa de limites de API decorrente. Evidentemente, o mesmo modelo para *embedding* dos *chunks* é o mesmo modelo utilizado para *embedding* das perguntas do usuário, que é o correto a se fazer.
- Modelo de LLM: `gemini-3.1-flash-lite`, melhor modelo “econômico” da plataforma Gemini API hoje para tarefas simples e diretas como RAG de pergunta-e-resposta. Conecta-se ao modelo via biblioteca `langchain-google-genai`.

- A pipeline de ingestão é feita ao executar o módulo `src/run_ingestion.py`. É feito:
 - o carregamento do text do documento PDF,
 - segmentação dos *chunks*,
 - o *embedding* do seu conteúdo,
 - e a inserção das passagens com seus *embeddings* associados dentro do banco PostgreSQL.

DIAGRAMA

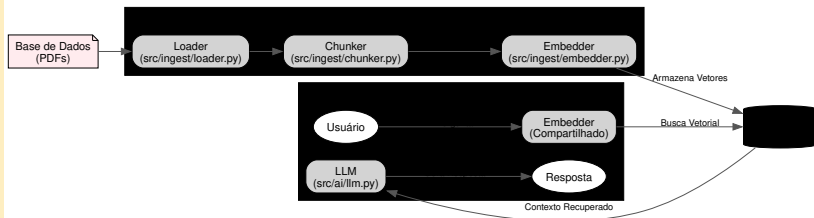


FIGURE 1: graphviz

RECUPERAÇÃO VETORIAL: ARMAZENAMENTO

DISPOSIÇÃO DO BANCO

O banco é otimizado, ao utilizar-se um esquema que utiliza quantização escalar com o tipo halfvec para tornar a ingestão mais rápida e eficiente, com perda mínima de precisão, tornando o desenvolvimento mais rápido sem sacrificar os resultados.

SCRIPT

```
CREATE TABLE chunk (  
    id INTEGER PRIMARY KEY  
        GENERATED ALWAYS AS IDENTITY,  
    snippet TEXT NOT NULL,  
    embedding halfvec(300) NOT NULL,  
    page INTEGER NOT NULL  
);
```

RECUPERAÇÃO VETORIAL: BUSCA

DESCRIÇÃO

- Para a busca semântica, uma similaridade de cosseno comum é aplicada, utilizando os operadores usuais fornecidos pela extensão PGVector.
- A busca semântica compara o embedding da pergunta do usuário com o que está no banco e retorna as passagens relevantes, de acordo com um valor de top-k correspondente. Foi estabelecido um valor de k=5 para os testes.

CONSULTA SQL

```
SELECT snippet, page, embedding <=> %s::halfvec  
AS distance  
FROM chunk  
ORDER BY distance  
LIMIT %s;
```

RECUPERAÇÃO VETORIAL: PERGUNTA

O formato do prompt é montado manualmente, com a pergunta do usuário no final. Foi feito dessa forma por ser uma boa prática: prompts no final com partes fixas no início se beneficiam do *prompt caching* fornecido pelas APIs. A seguir, o template de prompt montado pela equipe, em um formato XML, que costuma ser o formato mais bem entendido pelas LLMs, dado que delimita os limites de cada seção com tags, o que é satisfatório para a forma como a LLM processa texto: de forma linear.

RECUPERAÇÃO VETORIAL: PERGUNTA

<system>

O contexto a seguir vem de busca semântica.
Responda o usuário com base no contexto retornado.
Há a possibilidade do contexto não ser relevante,
dado que vem de uma busca semântica direta.

</system>

<context>

{"\n\n".join(search_results)}

</context>

<user>

{query}

</user>

VALIDAÇÃO OU MECANISMO HÍBRIDO

A validação do objeto retornado já é feita internamente pelo framework langchain, utilizado no trabalho. No presente trabalho, a implementação de validação é feita no caso de erros de solicitação à API do modelo. Em termos qualitativos, o formato do objeto a ser retornado para aplicação, correspondente à resposta da IA com as fontes, que consiste no output estruturado proposto no trabalho, aceita nenhuma fonte como resultado. Essa é a forma com que a aplicação lida com respostas possivelmente não encontradas. A seguir, o formato de objeto Pydantic esperado que o LLM retorne.

```
from pydantic import BaseModel, Field

class AIAnswer(BaseModel):
    content: str = Field(description="Sua resposta")
    sources: str | None = Field(description="""
    Todas as fontes para embasar a resposta""")
```

RESULTADOS EXPERIMENTAIS

EXPERIMENTO: ACERTOS X TOTAL ($k=5$)

Categoria	Acertos	Total
casos fáceis	3	6
casos médios	4	7
casos ambíguos	3	6
casos onde a base de conhecimento é insuficiente	5	6
casos que testam os limites da aplicacao	5	5
Total	20	30

RESULTADOS EXPERIMENTAIS

EXPERIMENTO: ACERTOS X TOTAL ($k=15$)

Categoria	Acertos	Total
casos fáceis	3	6
casos médios	6	7
casos ambíguos	3	6
casos onde a base de conhecimento é insuficiente	6	6
casos que testam os limites da aplicação	5	5
Total	23	30

- A equipe mostrou competência exemplar ao criar código de qualidade excepcional. Não houveram falhas lógicas ou execução detectadas pelo uso comum do programa. A pipeline de testes robusta cobre todo o ETL e pergunta ao LLM, e é uma prática que consiste em um exemplo a ser seguido por todos os demais.
- Em termos de qualidade de resultados, os embeddings gerados pela arquitetura word2vec do spaCy se mostraram como possível elo fraco no produto final. Ao mudar o valor de k de 5 para 15, houveram melhoras nos resultados (20×23), mas ainda se distanciando do ideal de 30.

- Considerou-se utilizar, sim, um modelo melhor de embedding (ex: `gemini-embedding-001` via Gemini API) para uma comparação melhor, porque é sabido que são grandemente superiores em qualidade, utilizando a arquitetura `transformers`. Mas como a especificação do trabalho menciona escolher um critério de comparação, ficamos por aqui. Isso é um trabalho de graduação, não uma tese de mestrado!

CONSIDERAÇÕES SOBRE USO DE IA

- O código experimental de comparativo entre valores *top-K* e de avaliação de desempenho foram feitos de forma assistida por agentes de IA, com base no código existente. O arquivo `./docs/protocolo_experimental.txt` mostra a íntegra da conversa.
- O arquivo `./docs/uso_ia_generativa.json` mostra a íntegra de algumas conversas e links de conversas usadas durante o desenvolvimento.
- O diagrama de arquitetura foi fornecido por uma IA, em código DOT (Graphviz).
- Todo o resto do desenvolvimento foi feito à mão, com parte do código fortemente pautada em cima de trabalhos anteriores similares feitos ao longo do semestre letivo dos discentes (em especial, da disciplina de Tópicos Especiais de Engenharia de *Software*). O mesmo vale para o relatório e slides, escritos em *Markdown*.

